Distributed Data Storage

1 Overview

To store a relation r in a distributed database, two main approaches are used:

- **Replication:** Maintaining several identical replicas (copies) of the relation at different sites.
- **Fragmentation:** Partitioning the relation into several fragments, each stored at a different site.

These can be combined: a relation can be partitioned into fragments, and each fragment can be replicated.

2 Data Replication

In **full replication**, a copy of relation r is stored at every site.

Pros and Cons of Replication

Advantages:

- **Availability:** If one site fails, r can be found at another site.
- **Increased Parallelism:** Multiple sites can process read-only queries in parallel.
- **Reduced Data Movement:** Minimizes data transfer if the replica is at the site of execution.

Disadvantages:

- **Update Overhead:** Updates must be propagated to all replicas, increasing complexity and cost.
- **Concurrency Control:** Managing simultaneous updates to multiple replicas is difficult.

*Note: Management is simplified using a **primary copy** scheme where one site is designated as the master for updates.*

3 Data Fragmentation

Fragmentation divides a relation r into fragments $r_1, r_2, \ldots, r_n$ that contain sufficient information to allow reconstruction.

3.1 Horizontal Fragmentation

Splits the relation by assigning each tuple of r to one or more fragments based on a selection predicate P_i.

- **Definition:** $r_i = \sigma_{P_i}(r)$
- **Reconstruction:** $r = r_1 \cup r_2 \cup \cdots \cup r_n$

Example: `account` **split by** `branch_name`

Branch	Acc No.	Balance
Hillside	A-305	500
Hillside	A-226	336
Hillside	A-155	62

$account_1 = \sigma_{\text{Hillside}}(account)$

Branch	Acc No.	Balance
Valleyview	A-177	205
Valleyview	A-402	10000
Valleyview	A-408	1123
Valleyview	A-639	750

$account_2 = \sigma_{\text{Valleyview}}(account)$

3.2 Vertical Fragmentation

Splits the relation schema R into subsets $R_1, R_2, \ldots, R_n$. To ensure losslessness, a **candidate key** (or a unique **tuple-id**) must be included in each fragment.

- **Definition:** $r_i = \Pi_{R_i}(r)$
- **Reconstruction:** $r = r_1 \bowtie r_2 \bowtie \ldots \bowtie r_n$

Vertical Fragmentation: `employee_info`

Branch	Customer	ID
Hillside	Lowman	1
Hillside	Camp	2
Valleyview	Camp	3
...	...	...

$deposit_1 = \Pi_{\text{branch, customer, id}}$

Acc No.	Balance	ID
A-305	500	1
A-226	336	2
A-177	205	3
...	...	...

$deposit_2 = \Pi_{\text{acc, balance, id}}$

3.3 Advantages of Fragmentation

- **Horizontal:** Allows parallel processing; places data where it is most frequently accessed.
- **Vertical:** Splits tuples to minimize access costs; `tuple-id` enables efficient joining; allows parallel processing on attributes.

4 Data Transparency

Transparency is the degree to which users are unaware of the details of how and where data is stored.

- **Fragmentation Transparency:** Users don't know how relations are partitioned.
- **Replication Transparency:** Users view data objects as logically unique.
- **Location Transparency:** System finds data given only the identifier.

4.1 Naming and Identification Criteria

1. Every data item must have a **system-wide unique name**.
2. It should be possible to **locate items efficiently**.
3. **Location changes** should be transparent.
4. Each site should be able to create new data items **autonomously**.

4.2 Implementation Strategies

- **Centralized Name Server:**
  - *Structure:* A central server assigns names and tracks locations.
  - *Disadvantages:* Bottleneck risk, single point of failure, limits autonomy.
- **Site Prefixes:**
  - *Structure:* Each site prefixes names with its ID (e.g., `site17.account`).
  - *Drawback:* Fails location transparency as the site is visible in the name.
- **Aliases:**
  - *Solution:* Users use simple names; site-specific mappings translate them to full names. This hides physical location and allows moving data without affecting users.

Transparency and naming strategies ensure a seamless user experience in complex distributed environments.